`LakeTemperature` Testing Software

Slides by Aidan Bensch and Sam Allen

Background

- We are testing a module of *E3SM,*[1] an environmental model. This module, `LakeTemperature`, is part of the greater *ELM* (*E3SM* Land Model) which was originally a fork of the *CLM* (Community Land Model).
- We are able to test specifically this module using *SPEL,*[2] a software that adds a *NetCDF*[3] interface for the model constants, inputs, and outputs of a module of *E3SM* and compiles that specific part of the model. The executable produced by this is named `elmtest.exe`.

1. “Energy Exascale Earth System Model.” *E3SM*, U.S. Department of Energy: Office of Science, n.d., e3sm.org/.

2. Schwartz, Peter. “SPEL_OpenACC.” *GitHub*, 16 Oct. 2025, github.com/peterdschwartz/SPEL_OpenACC.git.

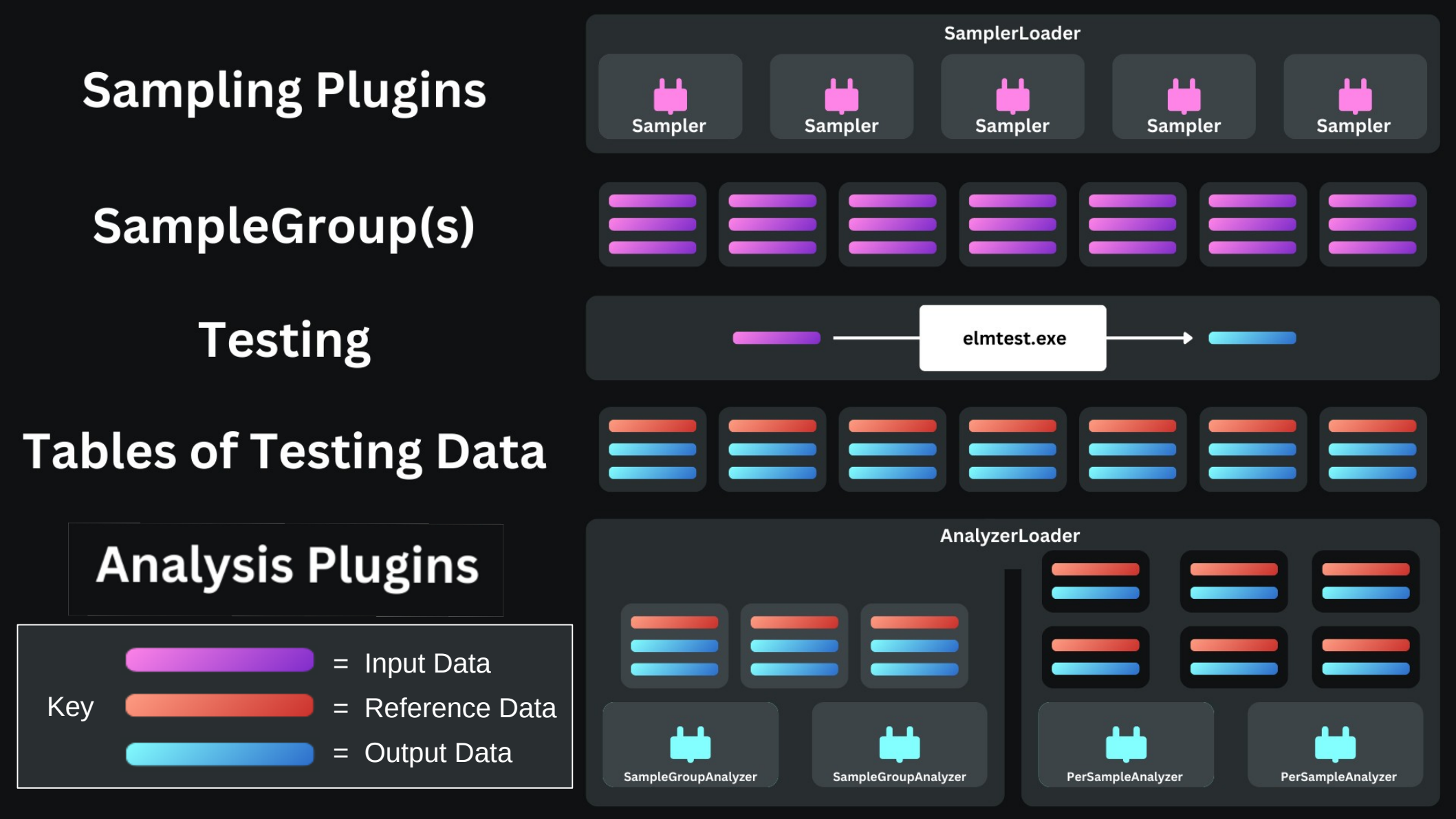
3. “NetCDF.” *NSF UNIDATA*, n.d., www.unidata.ucar.edu/software/netcdf.

Project Goals

- Develop a software that can systematically change the model constants used in the `LakeTemperature` model and allow us to visualize and observe patterns in how this affects its output.
- Apply testing techniques to try and find values for model constants that will cause physically impossible or otherwise erroneous model outputs. Apply assertions pertaining to the values of model constants, inputs, and outputs based around physical laws and other constraints provided by domain scientists.

Our Testing Solution

- Automates the process of modifying model constants in `spel-constants0001.nc` and the process of analyzing the resulting values in `fut-outputs0001.nc`
- Both the process of choosing values for each model constant and the process of analyzing the resulting model outputs is extendable through sampling plugins and analysis plugins respectively.


Sampling Plugins
SamplerLoader
Sampler
Sampler
Sampler
Sampler
Sampler
SampleGroup(s)
Testing
elmtest.exe
Tables of Testing Data
Analysis Plugins
AnalyzerLoader
Key
= Input Data
= Reference Data
= Output Data
SampleGroupAnalyzer
SampleGroupAnalyzer
PerSampleAnalyzer
PerSampleAnalyzer

Steps of Testing

Sample Generation

Using a Sampling Plugin to create samples
with unique values for the 25 scalar model constants

Running `elmtest.exe`

Running the program using the model constants
and recording the results

Analysis

Using an Analysis Plugin to analyze the results,
especially in comparison to reference data

Model Constants, Inputs, and Outputs

- 28 *model constants* control how the model behaves and manipulates data.
  - There are 3 array constants that, along with 8 scalar constants, do not have an observable effect on the outputs.
- 25 *model inputs* provide starting data for the model.
- 9 *model outputs* record data that the model has generated.
- Additional 8 variables serve as both model inputs and outputs.

Test Inputs: Model Constants (28)

Scalar Constants with Observable Outputs (17)	betavis, cnfac, cpliq, denh2o, dtime_mod, grav, hfus, lake_no_ed, lakepuddling, nlevgrnd, nlevlak, nlevsno, nlevsoi, thk_bedrock, tkwat, use_lch4, vkc
Scalar Constants without Observable Outputs (8)	cpice, denice, depthcrit, iulog, mixfact, pudz, tkair, tkice
Array Constants without Observable Outputs (3)	crop, iscft, percrop

Test Outputs: Model Inputs & Outputs (42)

Model Inputs Only (25)	col_es%t_grnd, col_pp%dz, col_pp%dz_lake, col_pp%z, col_pp%z_lake, col_pp%zi, col_pp&lakedepth, col_pp&snl, col_ws%h2osno, col_ws%h2osoi_ice, col_ws%h2osoi_liq, lakestate_vars%etal_col, lakestate_vars%ks_col, lakestate_vars%lake_raw_col, lakestate_vars%ws_col, soilstate_vars%csol_col, soilstate_vars%tkmg_col, soilstate_vars%tksatu_col, soilstate_vars%watsat_col, solarabs_vars%fsds_nir_d_patch, solarabs_vars%fsds_nir_i_patch, solarabs_vars%fsr_nir_d_patch, solarabs_vars%fsr_nir_i_patch, solarabs_vars%sabg_lyr_patch, solarabs_vars%sabg_patch
Model Inputs/ Outputs (8)	col_es%t_lake, col_es%t_soisno, lakestate_vars%lake_icefrac_col, lakestate_vars%lake_icethick_col, veg_ef%eflx_gnet, veg_ef%eflx_sh_grnd, veg_ef%eflx_sh_tot, veg_ef%eflx_soil_grnd
Model Outputs Only (9)	ch4_vars%grnd_ch4_cond_col, col_ef%errsoi, col_es%hc_soi, col_es%hc_soisno, col_wf%qflx_snofrz, col_ws%frac_iceold, lakestate_vars%betaprime_col, lakestate_vars%lakeresist_col, lakestate_vars%savedtke1_col

NetCDF

- Network Common Data Form (*NetCDF*) is a format for self-describing datasets. Each file is composed of “dimensions” and “variables.” Variables are multi-dimensional arrays, with the meaning of each dimension described in the file.

```
netcdf x_times_y {
dimensions:
        x = 5 ;
        y = 5 ;
variables:
        float x(x) ;
        float y(y) ;
        float z(x, y) ;
data:

 x = -2, -1, 0, 1, 2 ;

 y = -4, -2, 0, 2, 4 ;

 z =
  8, 4, -0, -4, -8,
  4, 2, -0, -2, -4,
  -0, -0, 0, 0, 0,
  -4, -2, 0, 2, 4,
  -8, -4, 0, 4, 8 ;
}
```

What is a Sample?

- A sample is a set of values for the model constants contained in `spel-constants0001.nc`.
- `elmtest.exe` is run once for each sample.

What is a Sample? (Illustration)

Model Constants (formerly lakeparams.txt)

```
betavis
0.0
za_lake
0.6
n2min
7.4999999999999993E-00
5
tdmax
277.0
pudz
0.0
mixfact
10.0
depthcrit
25.0
```

Example Sample (*JSON* used for illustration)

```
{
  "betavis": 0.0,
  "za_lake": 0.6,
  "n2min": 7.4999999999999993E-005,
  "tdmax": 277.0,
  "pudz": 0.0,
  "mixfact": 10.0,
  "depthcrit": 25.0
}
```

What Outputs are Received per Sample?

- Analysis plugins will receive the model outputs from the `spel-outputs0001.nc` file in the form of multidimensional `numpy` arrays, which range from one to three dimensions.

Outputs Received by Analysis Plugins (*JSON* for illustration)

```
{
  "veg_ef%eflx_soil_grnd": [0.0, ...],
  "veg_ef%eflx_gnet": [[0.0, ...], ...],
  ...
}
```

Sample Groups

- Represent a grouping of ordered samples
  - When the program is run on each sample in a sample group, the model data is returned in the same order as the samples were in.
- Having multiple sample groups is simply a convenient way of organizing similar samples.

Sample Group Illustration

Sample Group Data (*JSON* for illustration)

```
[
  {
    "betavis": 0.0,
    ...
  },
  {
    "betavis": 0.25,
    ...
  },
  {
    "betavis": 0.5,
    ...
  }
]
```

Ordered Output Data (*JSON* for illustration)

```
[
  {
    "lakestate_vars%betaprime_col": [0.0, ...],
    ...
  },
  {
    "lakestate_vars%betaprime_col": [0.25, ...],
    ...
  },
  {
    "lakestate_vars%betaprime_col": [0.5, ...],
    ...
  }
]
```

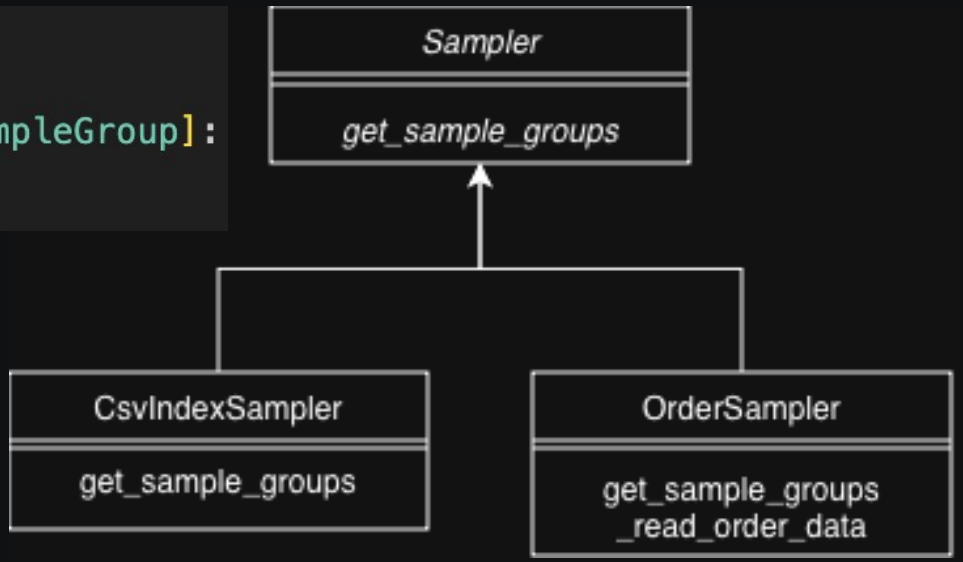
Plugins

- Our testing framework supports two types of plugins: sampling and analysis. Sampling plugins define values for model constants, and analysis plugins view the model outputs.
- Two sampling plugins are currently supported:
  - CSV Index Sampling, Order Sampling
- Four analysis plugins are currently supported:
  - Change Detection, Differences, Fault Finding, NetCDF4 Output

Sampling Plugin Overview

- Sampling plugins define a subclass of `Sampler` and return one or more `SampleGroups`.

```python
class Sampler(ABC):
    @abstractmethod
    def get_sample_groups(self) -> Mapping[str, SampleGroup]:
        pass
```



Our Sampling Plugins (2)

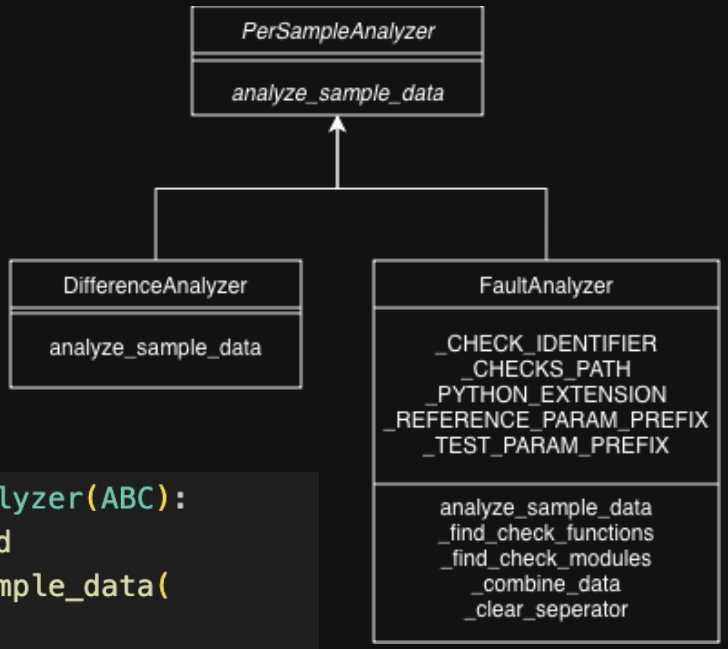
CSV Index Sampling	Creates samples using *CSV* files generated by *NIST*'s *ACTS*[1] tool by linearly interpolating between the values for each constant found in the ranges file
Order Sampling	Generates Order(s). Each Order reads instructions from a *JSON* file to generate samples using linear interpolation. Each Order will return one sample group.

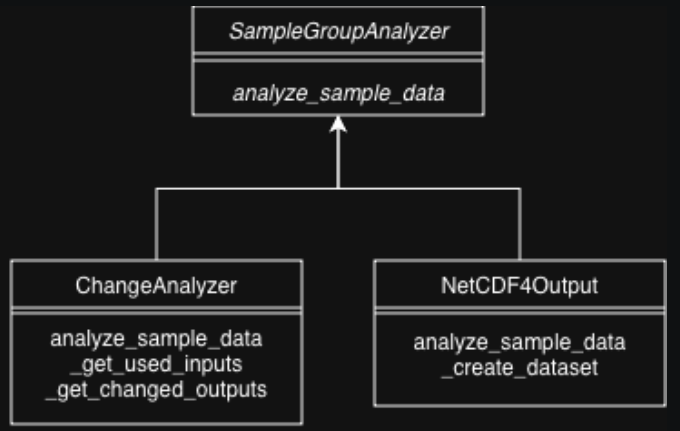
1. "Automated Combinatorial Testing for Software (ACTS)." *National Institute of Standards and Technology*, 20 July 2016, csrc.nist.rip/groups/SNS/acts/.

Analysis Plugin Overview

- Analysis plugins can either be “per sample” or “sample group” analysis plugins.
- “Per sample” plugins define a subclass of `PerSampleAnalyzer`, whose method is called each time `elmtest.exe` is run, and which receives the model outputs produced by that specific sample.
- “Sample group” plugins define a subclass of `SampleGroupAnalyzer`, whose method is called after all samples in a group are run through `elmtest.exe`, and which receives all model outputs from each.

Analysis Plugin Overview (Cont.)






```python
class PerSampleAnalyzer(ABC):
    @abstractmethod
    def analyze_sample_data(
        self,
        sample: Sample,
        reference_data: ModelData,
        test_data: ModelData,
    ) -> tuple[str, FileSystemTree]:
        pass
```

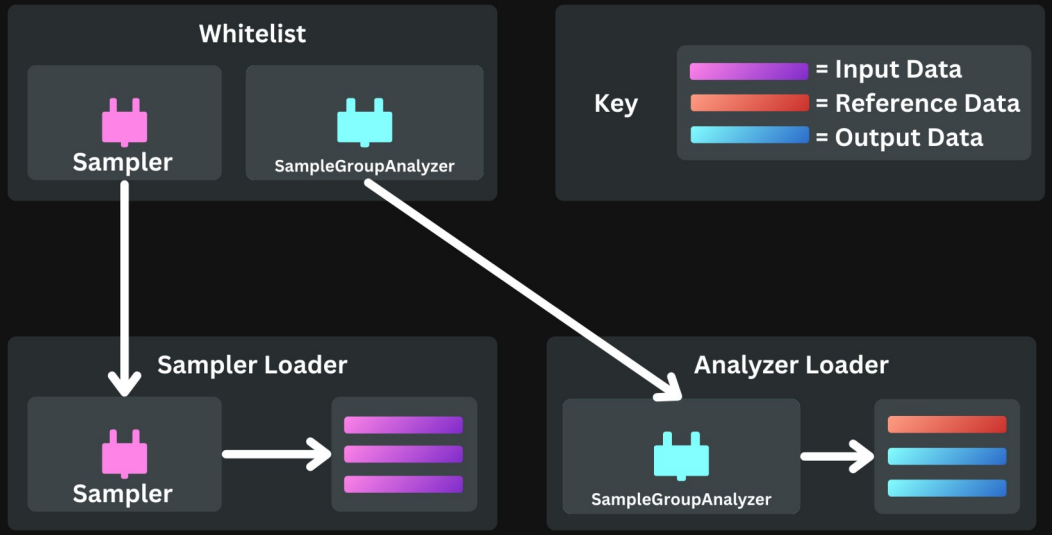
```python
class SampleGroupAnalyzer(ABC):
    @abstractmethod
    def analyze_sample_data(
        self,
        sample_group: SampleGroup,
        reference_data: ModelData,
        sample_data: list[ModelData],
    ) -> tuple[str, FileSystemTree]:
        pass
```

Our Analysis Plugins (4)

Per Sample Analyzers	Differences	Finds differences between values in the reference data and the current sample's data, recording the indices of the difference, the two differing values, and the difference between them
	Fault Finding	Uses functions defined by the plugin to make assertions about values in the sample data, similar to a unit testing framework but specialized to test model output values
Sample Group Analyzers	Change Detection	Identifies which model inputs and outputs differ from the reference
	NetCDF4 Output	Stores each sample group as an individual *NetCDF* file, to be used for graphing changes based on our samples

Plugin Loader

- Is an abstract class that loads whitelisted plugins



Example Whitelist

```
{
    "sampling": [
        "order_sampling"
    ],
    "analysis": [
        "fault_finding"
    ]
}
```

Sampler Loader

- Defines a subclass of Plugin Loader
- Sampler Loader loads the whitelisted Sampling Plugins.
- It then returns the generated Sample Groups mapped to their names in a dictionary.

```python
class SamplerLoader(PluginLoader):
    def sample_from_plugins(self) -> dict[str, SampleGroup]:
        sample_groups: dict[str, SampleGroup] = {}
        samplers = self._plugin_objects[Sampler]

        for plugin_name in samplers:
            sampler = samplers[plugin_name]
            event_bus.fire_event(
                Event.BEGAN_SAMPLING_FROM_PLUGIN,
                plugin_name=plugin_name
            )

            try:
                sampler_sample_groups = sampler.get_sample_groups()
            except Exception as error:
                event_bus.fire_event(
                    Event.SAMPLING_FROM_PLUGIN_FAILURE,
                    plugin_name=plugin_name,
                    reason=error
                )
                continue

            self._add_new_sample_groups(
                sample_groups,
                sampler_sample_groups,
                plugin_name
            )

            group_sample_counts = {
                group_name: len(sampler_sample_groups[group_name])
                for group_name in sampler_sample_groups
            }

            event_bus.fire_event(
                Event.SAMPLING_FROM_PLUGIN_SUCCESS,
                plugin_name=plugin_name,
                group_sample_counts=group_sample_counts
            )

        return sample_groups
```

Analyzer Loader

- Defines a subclass of Plugin Loader
- Analyzer Loader instantiates the Analysis Plugins and gives the Analysis Plugins the model data they need.

```python
class AnalyzerLoader(PluginLoader):
    def run_sample_analysis(
            self,
            sample: Mapping[str, float],
            reference_data: Mapping[str, Sequence[float]],
            test_data: Mapping[str, Sequence[float]]
    ) -> None:
        for plugin_name, analyzer in self._plugin_objects[PerSampleAnalyzer].items():
            event_bus.fire_event(
                Event.BEGAN_SAMPLE_ANALYSIS_WITH_PLUGIN,
                plugin_name=plugin_name
            )

            try:
                console_out, file_out = analyzer.analyze_sample_data(
                    sample, reference_data, test_data
                )
                event_bus.fire_event(
                    Event.SAMPLE_ANALYSIS_WITH_PLUGIN_SUCCESS,
                    plugin_name=plugin_name,
                    console_output=console_out,
                    file_output=file_out
                )
            except Exception as error:
                event_bus.fire_event(
                    Event.SAMPLE_ANALYSIS_WITH_PLUGIN_FAILURE,
                    plugin_name=plugin_name,
                    reason=error
                )
```

```python
class AnalyzerLoader(PluginLoader):
    def run_group_analysis(
            self,
            sample_group: SampleGroup,
            reference_data: ModelData,
            sample_data: list[ModelData],
    ) -> None:
        for plugin_name, analyzer in self._plugin_objects[SampleGroupAnalyzer].items():
            event_bus.fire_event(
                Event.BEGAN_GROUP_ANALYSIS_WITH_PLUGIN,
                plugin_name=plugin_name
            )

            try:
                console_out, file_out = analyzer.analyze_sample_data(
                    sample_group, reference_data, sample_data
                )
                event_bus.fire_event(
                    Event.GROUP_ANALYSIS_WITH_PLUGIN_SUCCESS,
                    plugin_name=plugin_name,
                    console_output=console_out,
                    file_output=file_out
                )
            except Exception as error:
                event_bus.fire_event(
                    Event.GROUP_ANALYSIS_WITH_PLUGIN_FAILURE,
                    plugin_name=plugin_name,
                    reason=error
                )
```

Configuration

- The `LakeTemperature` Testing Software and its plugins are controlled by configuration files.
- These files are loaded by different parts of the program and allow it to be adapted to different models and testing requirements.

Example Configurations

Main Configurations

```
[AppPaths]
app_root: .

[Model]
binary: model/elmtest
parameters: model/spel-constants0001.nc
parameter_defaults: model/spel-constant-defaults0001.nc
reference_output: model/spel-outputs0001.nc
test_output: model/fut-outputs0001.nc
args: 1
```

Plugin Whitelist Configurations

```
{
    "sampling": [
        "order_sampling"
    ],
    "analysis": [
        "fault_finding"
    ]
}
```

Output Configurations

Files

```
[Directories]
samples: testing_output/samples/
errors: testing_output/errors/
analysis: testing_output/analysis/
```

Logs

```
[Directories]
log_directory: logs/

[Settings]
keep_logs: 5
```

Sampling Plugin Configurations

NetCDF Cases

```
[Paths]
group_directory: plugin_inputs/netcdf_cases/
```

Order Sampling

```
[Paths]
range_path: model/FUT_lake_range.txt
csv_index_files_directory: plugin_inputs/csv_index_files/
```

Analysis Plugin Configurations

Fault Finding

```
[Paths]
checks_directory: plugin_inputs/fault_checks/
```

NetCDF4 Output

```
[Format]
store_as_samples: yes
```

Example Run

```
root@f302b01b4b69:/app# mtf config/
Successfully loaded binary.

Loading sampling plugins:
  [Mtf Order Sampling]: Loaded

Loaded 1 sampling plugins successfully.

Loading analysis plugins:
  [Mtf Fault Finding]: Loaded

Loaded 1 analysis plugins successfully.

Sampling from plugins:
  [Mtf Order Sampling]:
    betavis_min_to_max: 11 samples
    Total: 1 groups, 11 samples

  Grand Total: 1 groups, 11 samples
```

```
Time estimated: 0:0:0:8
Would you like to continue? (Yes/No)
Yes

Group 1 / 1: betavis_min_to_max

Sample 1 / 11:
  betavis: 0.0

Binary ran successfully.

Sample Analysis:
  [Mtf Fault Finding]:
    Results (39 checks run):
      [PASSED]: 32 checks
      [SKIPPED]: 5 checks
      [FAILED]: 2 checks

    [FAILED]: check_imelt_uses_valid_enum_values
    [FAILED]: check_melting_latent_heat
```

```
Sample 11 / 11:
  betavis: 1.0

Binary ran successfully.

Sample Analysis:
  [Mtf Fault Finding]:
    Results (39 checks run):
      [PASSED]: 32 checks
      [SKIPPED]: 5 checks
      [FAILED]: 2 checks

    [FAILED]: check_imelt_uses_valid_enum_values
    [FAILED]: check_melting_latent_heat


Testing completed.
```

Use Case Diagram